

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №30

Тема: Жадібні алгоритми

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати принцип жадібних алгоритмів (локально оптимальний вибір на кожному кроці), навчитися розпізнавати задачі, де жадібна стратегія дає оптимальний результат (вибір активностей, неперервний рюкзак, розмін монет канонічними номіналами, кодування Гаффмана), та ті, де вона помиляється, порівнюючи з динамічним програмуванням і повним перебором.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Принцип жадібності. Жадібний алгоритм будує розв'язок крок за кроком, на кожному кроці обираючи варіант, що виглядає **найкращим у цей момент** (локально оптимальний), і ніколи не переглядає свій вибір. Такий підхід простий і швидкий (зазвичай $O(n \log n)$ через сортування), але гарантує глобальний оптимум не для всіх задач.

2. Умови коректності. Жадібний алгоритм оптимальний, якщо задача має: (а) **властивість жадібного вибору** — існує оптимальний розв'язок, що містить жадібний вибір першого кроку; (б) **оптимальну підструктуру** — після жадібного вибору залишається задача того самого типу. Типова схема доведення — **метод обміну**: будь-який оптимальний розв'язок можна перетворити так, щоб він містив жадібний вибір, не погіршивши результат.

3. Класичні задачі та жадібні стратегії:

Задача	Жадібний критерій	Оптимальність
Вибір максимальної кількості неперетинних активностей (відрізків)	брати активність, що закінчується найраніше й не перетинається з вибраними	так (доведено)
Неперервний (дробовий) рюкзак	брати предмети за спаданням питомої цінності v/w ; останній — частково	так
Розмін монет канонічної системи (наприклад, 1, 2, 5, 10, 20, 50...)	найбільша монета, що не перевищує залишку	так
Розмін монет довільної системи (наприклад, 1, 3, 4)	найбільша монета, що не перевищує залишку	ні (приклад: $6 = 4 + 1 + 1$, а краще $3 + 3$)
Рюкзак 0/1	за спаданням v/w	ні — потрібне ДП
Кодування Гаффмана	двічі об'єднувати два найрідкісніші символи в піддерево	так (мінімальна довжина префіксного коду)
Мінімальне остовне дерево (Крускал, Прим)	брати найдешевше ребро без утворення циклу	так

4. Кодування Гаффмана. Кожному символу присвоюється **префіксний** (жоден код не є початком іншого) двійковий код змінної довжини: частіші символи — коротші коди. Алгоритм: помістити символи в чергу з пріоритетом за частотою; доки в черзі більше одного елемента — вилучити два елементи з найменшими частотами й додати новий вузол із сумою частот. Кодом символу є шлях від кореня до листка (ліворуч — «0», праворуч — «1»).

5. Як відрізнити задачу, розв'язну жадібно. Спробувати довести жадібний вибір методом обміну або знайти **контрприклад** на малих даних. Практичний спосіб — порівняти результат жадібного алгоритму з результатом повного перебору (або ДП) на випадкових малих тестах: якщо знайдено розбіжність — жадібна стратегія для цієї задачі некоректна.

Хід роботи

Завдання 1. Ручний розв'язок: вибір активностей

Дано 8 занять із часом початку та завершення: A(1; 4), B(3; 5), C(0; 6), D(5; 7), E(3; 9), F(5; 9), G(6; 10), H(8; 11). Одночасно проводити заняття в одній аудиторії не можна. Вибрати максимальну кількість занять.

Порядок виконання:

1. Впорядкуйте заняття за часом завершення й запишіть таблицю «заняття — початок — кінець».
2. Застосуйте жадібний критерій: беріть перше заняття; кожне наступне, початок якого не раніше кінця останнього вибраного; запишіть таблицю кроків «заняття – початок $\geq$ кінця вибраного? – рішення».
3. Запишіть вибрану множину та її розмір.
4. Спробуйте альтернативні критерії: (а) за раннім початком, (б) за найкоротшою тривалістю; для кожного знайдіть результат та порівняйте розмір множини з відповіддю за критерієм «найраніше завершення».
5. Побудуйте контрприклад для критерію «найраніший початок»: набір занять, де цей критерій дає лише одне заняття, хоча можна вибрати кілька.

Завдання 2. Ручний розв'язок: рюкзак і розмін монет

Порівняти жадібну стратегію для неперервного та дискретного (0/1) рюкзака, а також для двох систем монет.

Порядок виконання:

1. Рюкзак місткістю 50; предмети (вага; цінність): (10; 60), (20; 100), (30; 120). Обчисліть питому цінність v/w кожного предмета.
2. Розв'яжіть **неперервну** задачу жадібно: беріть предмети за спаданням v/w ; останній — частково, скільки влізе; обчисліть загальну цінність.
3. Розв'яжіть **дискретну** задачу 0/1 жадібно (за v/w , без дроблення) і порівняйте з оптимальним розв'язком, знайденим повним перебором усіх $2^3 = 8$ підмножин. Зробіть висновок.
4. Розмін 63 грн монетами 1, 2, 5, 10, 20, 50: виконайте жадібний розмін і запишіть склад монет; переконайтесь, що вибір оптимальний.
5. Розмін 6 монетами 1, 3, 4: виконайте жадібний розмін, а потім знайдіть краще розв'язання; поясніть, чому властивість жадібного вибору тут не виконується.

Завдання 3. Програмна реалізація жадібних алгоритмів

Реалізувати мовою Python жадібні алгоритми вибору активностей, неперервного рюкзака та розміну монет; порівняти розмін монет із оптимальним розв'язком ДП.

Порядок виконання:

1. Напишіть функцію `select_activities(acts)`, що сортує заняття за часом завершення й повертає список вибраних.
2. Напишіть функцію `fractional_knapsack(items, W)`, що повертає максимальну цінність та частки взятих предметів.
3. Напишіть функцію `greedy_change(coins, amount)` для розміну жадібним методом (монети беруть за спаданням) та функцію `dp_change(coins, amount)` — оптимальний розмін динамічним програмуванням (практична робота №29).
4. Перевірте функції на даних завдань 1, 2 (результати мають збігатися з ручними).
5. Для систем монет (1, 2, 5, 10, 20, 50) та (1, 3, 4) знайдіть усі суми від 1 до 100, для яких жадібний розмін не є оптимальним; виведіть кількість таких сум і перші з них.

Приклад реалізації мовою Python:

python

```
def select_activities(acts):
    chosen, last_end = [], -1
    for name, (start, end) in sorted(acts.items(), key=lambda kv: kv[1][1]):
        if start >= last_end:
            chosen.append(name)
            last_end = end
    return chosen

def fractional_knapsack(items, W):
    order = sorted(items, key=lambda it: it[1] / it[0], reverse=True)
    total, left, parts = 0.0, W, []
    for w, v in order:
        take = min(w, left)
        total += v * take / w
        parts.append((w, v, take / w))
        left -= take
        if left == 0:
            break
    return total, parts

def greedy_change(coins, amount):
    used = []
    for c in sorted(coins, reverse=True):
        while amount >= c:
            amount -= c
            used.append(c)
    return used if amount == 0 else None

def dp_change(coins, amount):
    INF = float("inf")
    dp = [0] + [INF] * amount
    for s in range(1, amount + 1):
        for c in coins:
            if c <= s:
                dp[s] = min(dp[s], dp[s - c] + 1)
    return dp[amount]

acts = {"A": (1, 4), "B": (3, 5), "C": (0, 6), "D": (5, 7),
        "E": (3, 9), "F": (5, 9), "G": (6, 10), "H": (8, 11)}
print("Вибрані заняття:", select_activities(acts))
```

```
total, parts = fractional_knapsack([(10, 60), (20, 100), (30, 120)], 50)
print(f"Неперервний рюкзак: цінність {total:.1f}; частки: {[round(p[2], 2) for p in parts]}")

print("Розмін 63 грн:", greedy_change((1, 2, 5, 10, 20, 50), 63))
print("Розмін 6 (1, 3, 4):", greedy_change((1, 3, 4), 6), "; оптимально монет:",
      dp_change((1, 3, 4), 6))

for system in ((1, 2, 5, 10, 20, 50), (1, 3, 4)):
    bad = [s for s in range(1, 101) if len(greedy_change(system, s)) !=
           dp_change(system, s)]
    print(f"Система {system}: сум із неоптимальним жадібним розміном — {len(bad)};
          перші: {bad[:6]}")
```

Результат виконання програми:

```
Вибрані заняття: ['A', 'D', 'H']
Неперервний рюкзак: цінність 240.0; частки: [1.0, 1.0, 0.67]
Розмін 63 грн: [50, 10, 2, 1]
Розмін 6 (1, 3, 4): [4, 1, 1] ; оптимально монет: 2
Система (1, 2, 5, 10, 20, 50): сум із неоптимальним жадібним розміном — 0; перші: []
Система (1, 3, 4): сум із неоптимальним жадібним розміном — 24; перші: [6, 10, 14, 18,
22, 26]
```

Висновок за результатами: жадібний вибір за критерієм «найраніше завершення» дає три заняття A, D, H — стільки ж, скільки і ручний розв'язок. Для неперервного рюкзака отримано цінність 240 (предмети 1 і 2 повністю та 2/3 предмета 3), а жадібний розв'язок дискретної задачі (предмети 1 і 2) дає лише 160 при оптимумі 220 (предмети 2 і 3) — тож для рюкзака 0/1 потрібне ДП. Для канонічної системи монет (1, 2, 5, 10, 20, 50) жадібний розмін завжди оптимальний, а для системи (1, 3, 4) програма знаходить сум із неоптимальним результатом (перша з них — 6). Це підтверджує, що коректність жадібної стратегії залежить від структури задачі й має доводитись або перевірятись експериментально.

Завдання 4. Кодування Гаффмана

Побудувати вручну та програмно оптимальний префіксний код для рядка «abracadabra», оцінити ступінь стиснення.

Порядок виконання:

1. Підрахуйте частоти символів рядка «abracadabra» (11 символів) і запишіть таблицю «символ — частота».
2. Побудуйте вручну дерево Гаффмана: на кожному кроці об'єднуйте два вузли з найменшими частотами; записуйте кроки об'єднання та частоти нових вузлів.
3. Присвойте символам коди (ліва гілка — «0», права — «1») і запишіть таблицю кодів. Перевірте, що жоден код не є префіксом іншого.
4. Обчисліть довжину закодованого повідомлення в бітах і порівняйте з рівномірним кодуванням (3 біти на символ, оскільки символів 5) та 8-бітовим ASCII; обчисліть коефіцієнт стиснення.
5. Напишіть програму побудови дерева Гаффмана за допомогою `heapq`, виведіть таблицю кодів, закодуйте рядок і декодуйте його назад; перевірте збіг із початковим рядком.
6. Знайдіть ентропію джерела $H = -\sum p_i \log_2 p_i$ і порівняйте її із середньою довжиною коду Гаффмана.

Приклад реалізації мовою Python:

```
python
import heapq
import math
from collections import Counter
```

```

def huffman_codes(text):
    freq = Counter(text)
    heap = [[w, i, sym] for i, (sym, w) in enumerate(sorted(freq.items()))]
    heapq.heapify(heap)
    counter = len(heap)
    while len(heap) > 1:
        a = heapq.heappop(heap)
        b = heapq.heappop(heap)
        heapq.heappush(heap, [a[0] + b[0], counter, (a, b)])
        counter += 1
    codes = {}

    def walk(node, prefix):
        payload = node[2]
        if isinstance(payload, str):
            codes[payload] = prefix or "0"
        else:
            walk(payload[0], prefix + "0")
            walk(payload[1], prefix + "1")

    walk(heap[0], "")
    return codes, freq

def encode(text, codes):
    return "".join(codes[ch] for ch in text)

def decode(bits, codes):
    inverse = {code: ch for ch, code in codes.items()}
    result, cur = [], ""
    for b in bits:
        cur += b
        if cur in inverse:
            result.append(inverse[cur])
            cur = ""
    return "".join(result)

text = "abracadabra"
codes, freq = huffman_codes(text)
print("Символ частота код")
for sym in sorted(codes, key=lambda s: (len(codes[s]), s)):
    print(f" {sym} {freq[sym]:>2} {codes[sym]}")

bits = encode(text, codes)
print("\nЗакодований рядок:", bits)
print("Довжина:", len(bits), "біт; рівномірний код (3 біти):", 3 * len(text), "біт; ASCII:", 8 * len(text), "біт")
print("Декодування збігається з оригіналом:", decode(bits, codes) == text)
print(f"Коефіцієнт стиснення відносно ASCII: {8 * len(text) / len(bits):.2f}")

n = len(text)
entropy = -sum(c / n * math.log2(c / n) for c in freq.values())
avg_len = len(bits) / n
print(f"Ентропія H = {entropy:.4f} біт/символ; середня довжина коду = {avg_len:.4f} біт/символ")

```

Результат виконання програми:

Символ	частота	код
a	5	0
b	2	110
c	1	100
d	1	101
r	2	111

Закодований рядок: 01101110100010101101110

Довжина: 23 біт; рівномірний код (3 біти): 33 біт; ASCII: 88 біт
Декодування збігається з оригіналом: True
Коефіцієнт стиснення відносно ASCII: 3.83
Ентропія $H = 2.0404$ біт/символ; середня довжина коду = 2.0909 біт/символ

Висновок за результатами: для рядка «abracadabra» частіші символи отримують коротші коди (символ «a» трапляється 5 разів і отримує код довжиною 1), а весь рядок кодується у 23 біт — проти 33 біт при рівномірному 3-бітовому кодуванні та 88 біт в ASCII. Декодування відновлює початковий рядок, оскільки код префіксний. Середня довжина коду (2.091 біт/символ) лише трохи перевищує ентропію джерела (2.040) — код Гаффмана оптимальний серед префіксних, але не може бути коротшим за ентропію.

Контрольні питання

1. У чому полягає суть жадібного алгоритму? Чим він відрізняється від повного перебору та динамічного програмування?
2. Що таке властивість жадібного вибору й оптимальна підструктура? Як використовується метод обміну в доведеннях?
3. Який критерій забезпечує оптимальність у задачі вибору активностей? Наведіть контрприклад для критерію «найраніший початок».
4. Чому неперервний рюкзак розв'язується жадібно, а рюкзак 0/1 — ні?
5. За яких умов жадібний розмін монет оптимальний? Наведіть систему монет, для якої він помиляється.
6. Опишіть алгоритм Гаффмана. Що таке префіксний код і чому він дозволяє однозначне декодування?
7. Яка складність алгоритму Гаффмана та вибору активностей?
8. Як практично перевірити коректність жадібного алгоритму для нової задачі?

Зміст звіту

1. Тема, мета роботи.
2. Ручні розв'язки задач та дерево Гаффмана (завдання 1, 2, 4).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Висновки.